

Relatório seminário

Disciplina: Tópicos Avançados de Redes de Computadores

Tema: Roteamento baseado em intenção em linguagem P4

Grupo: Eduardo Braga, Henrique Franca, Isabela Medeiros, Júlia Vilela, Rafael Angelim e Reuel Bezerra

1. Fundamentação Teórica

Redes de computadores tradicionais: Contextualização sobre as redes de computadores tradicionais, apresentando seu funcionamento básico e a organização dos dispositivos em diferentes topologias. É discutido como o crescimento das redes aumenta sua complexidade e torna mais difícil sua configuração e gerenciamento. Esse problema será exemplificado por meio de um objetivo de alto nível, como priorizar o tráfego de videoconferências em relação a downloads, mostrando que um objetivo aparentemente simples precisa ser traduzido em diversas configurações, como identificação de tráfego, QoS, prioridades, filas, políticas e rotas.

Arquitetura tradicional de dispositivos de rede: Visualização da arquitetura tradicional de um switch destacando a divisão conceitual entre plano de gerenciamento, plano de controle e plano de dados. O plano de gerenciamento está relacionado à configuração e ao monitoramento dos equipamentos; o plano de controle determina ou influencia as decisões utilizadas no encaminhamento; e o plano de dados realiza efetivamente o encaminhamento dos pacotes.

SDN (Software-Defined Networking): Introduziremos o conceito de redes definidas por software, destacando a separação entre o plano de controle e o plano de dados e o papel do controlador SDN.

Protocolo OpenFlow: Apresentaremos o protocolo utilizado na comunicação entre o controlador e os dispositivos compatíveis. Será explicada a utilização das tabelas de fluxo (flow tables) e do modelo Match-Action, no qual determinadas características dos pacotes são comparadas com regras e associadas a ações, como encaminhar um pacote por determinada porta.

Limitações do OpenFlow: Apresentaremos algumas limitações do OpenFlow, principalmente relacionadas à flexibilidade do plano de dados, aos campos e cabeçalhos suportados e à dificuldade de introduzir novos protocolos e formas de processamento, o que acabou motivando a termos um plano de dados programável.

PISA (Protocol Independent Switch Architecture): Arquitetura utilizada para descrever o comportamento do plano de dados programável. Será explicado o fluxo básico de processamento de um pacote, desde sua entrada e interpretação pelo parser, passando pelas etapas de Match-Action, até sua reconstrução pelo deparser e saída do dispositivo.

Linguagem P4 (Programming Protocol-independent Packet Processors): Introduziremos a linguagem P4 utilizada para descrever o comportamento do plano de dados. Será explicado o fluxo básico de processamento de um pacote, desde sua entrada e interpretação pelo parser, passando pelas etapas de Match-Action, até sua reconstrução pelo deparser e saída do dispositivo.

Principais elementos da linguagem P4: Será abordado tipos de dados, headers, structures, operadores, expressões, parsers programáveis, tabelas e ações. Exemplos de código serão utilizados para demonstrar como cabeçalhos como Ethernet e IPv4 podem ser definidos e extraídos e como tabelas utilizam chaves para realizar operações de Match-Action.

Por fim, será mostrado o processo que leva um **programa P4 até o dispositivo de rede**, envolvendo o arquivo **.p4**, sua compilação e o dispositivo-alvo. Nesse contexto, será apresentado o **P4Runtime**, uma API de plano de controle utilizada para controlar elementos do plano de dados definidos pelos programas P4.

Bloco 2

Intenção (Intent): Será introduzido o conceito de intenção como uma declaração de alto nível do resultado desejado, sem especificar detalhadamente os passos necessários para alcançá-lo. Será apresentada a diferença entre a abordagem imperativa, na qual o operador determina “como fazer”, e a abordagem declarativa, utilizada no roteamento baseado em intenção, na qual é especificado “o que se deseja” e o sistema determina como alcançar esse resultado.

Exemplos de intenção em redes: Serão apresentados exemplos de políticas de alto nível relacionadas à segurança, qualidade de serviço e restrições de roteamento. Entre os exemplos estão o isolamento do tráfego de determinado departamento, a definição de limites de latência para videoconferências e a restrição da passagem de tráfego sensível por determinados enlaces ou regiões. Esses exemplos demonstram como uma intenção pode expressar o comportamento desejado sem especificar diretamente endereços IP, VLANs ou tabelas de roteamento.

Tradução de intenções: Será discutida a dificuldade de transformar uma intenção de alto nível em configurações concretas da infraestrutura de rede. Para realizar essa tradução, o sistema precisa considerar informações como a topologia, o estado atual dos enlaces e dispositivos e as capacidades disponíveis, além de possuir mecanismos de automação capazes de converter as políticas declaradas em regras aplicáveis ao plano de dados.

Intent-Based Networking (IBN): Será apresentado o ciclo de vida de uma intenção em uma arquitetura IBN, composto pelas etapas de declaração, tradução, validação, ativação e garantia/monitoramento. Após a declaração da intenção pelo operador, o sistema realiza sua tradução para configurações concretas, verifica possíveis inconsistências, aplica as configurações à rede e monitora continuamente se o comportamento obtido permanece de acordo com a intenção especificada.

Funcionamento prático do IBN: Será explicado o funcionamento do IBN como um ciclo contínuo de aplicação, monitoramento e adaptação da rede. Serão abordados mecanismos como telemetria em tempo real, tradução e verificação, automação e o possível suporte de

técnicas de Inteligência Artificial e Machine Learning para análise de métricas, identificação de gargalos e auxílio na otimização das configurações da rede.

Relação entre IBN e SDN: Será apresentada a diferença e a relação entre os dois conceitos. Enquanto o SDN fornece mecanismos de controle programável da infraestrutura, permitindo centralização lógica do controle e instalação de regras e fluxos, o IBN acrescenta uma camada de abstração baseada em objetivos de alto nível, responsável pela tradução, automação, validação e monitoramento contínuo das intenções.

Integração entre IBN, SDN e P4: Será demonstrado como esses conceitos podem atuar de maneira integrada. O IBN recebe e traduz o objetivo de alto nível; o controlador SDN utiliza as instruções resultantes para controlar a infraestrutura; e os dispositivos programáveis em P4 oferecem flexibilidade no plano de dados para executar as políticas necessárias.

Roteamento baseado em intenção: Será apresentada a mudança de paradigma em relação ao roteamento convencional, no qual a escolha do caminho pode deixar de considerar apenas métricas tradicionais e passar a incorporar políticas explícitas, como requisitos de QoS, isolamento, engenharia de tráfego e restrições de segurança. Dessa forma, o caminho utilizado pode ser resultado da tradução de uma intenção de alto nível para regras concretas de rede.

Requisitos técnicos para o roteamento baseado em intenção: Serão discutidos os requisitos necessários para executar as políticas resultantes das intenções, incluindo maior flexibilidade em operações Match-Action, possibilidade de considerar diferentes campos dos pacotes e suporte a processamento condicional e políticas mais complexas diretamente no plano de dados. Nesse contexto, será estabelecida a relação com a programabilidade proporcionada pelo P4.

Limitações do hardware de rede tradicional: Serão apresentadas as dificuldades de executar políticas mais flexíveis utilizando hardware com conjuntos predefinidos de campos e ações. Será discutido como essas limitações podem restringir a tradução de determinadas intenções e aumentar a dependência do plano de controle para decisões que não podem ser realizadas diretamente pelo dispositivo.

P4 como suporte ao IBN: Será mostrado como a programabilidade do plano de dados pode fornecer maior flexibilidade para implementar as políticas geradas pelo sistema de intenção. Serão abordadas a criação de tabelas, campos e ações customizadas, a utilização do P4Runtime para reconfiguração em tempo de execução e a possibilidade de executar mecanismos como telemetria, marcação de pacotes e decisões relacionadas a QoS e segurança diretamente no plano de dados.

Arquitetura conceitual integrada: Será apresentada uma visão completa da arquitetura, composta pela camada de intenção, responsável pela declaração das políticas; camada de tradução, responsável pela conversão das intenções em regras concretas; camada de controle,

utilizando um controlador/orquestrador; camada de dados, formada pelos dispositivos programáveis em P4; e um ciclo de retroalimentação responsável por verificar se o comportamento observado continua de acordo com a intenção definida.

Por fim, serão discutidos alguns desafios ainda existentes, como possíveis ambiguidades e conflitos entre diferentes intenções, a necessidade de verificar se as configurações geradas realmente implementam a intenção declarada, as limitações físicas do plano de dados e os problemas de escalabilidade diante de múltiplas intenções e alterações dinâmicas da rede.

2. Estado da arte

2.1 P4I/O e Araucaria

Dois trabalhos ilustram essa tradução de intenção para código P4: o P4I/O, de [Riftadi e Kuipers \(2019\)](#), e o Araucaria, de [Parizotto et al. \(2024\)](#).

P4I/O: o código P4 é, por natureza, estático. Uma vez compilado e instalado, alterar o comportamento do switch normalmente exige reescrever o programa à mão e recompilar. O P4I/O propõe uma linguagem de intenção própria, a IDL (Intent Definition Language), que descreve o objetivo desejado em um formato fixo; um componente do sistema recebe essa intenção por HTTP, seleciona o template P4 correspondente em um repositório de templates prontos, injeta os parâmetros declarados e monta um programa P4 completo, compilado com o compilador padrão p4c. Um exemplo do próprio artigo é a intenção `dropHeavyHitters threshold(more, 50)`, em que o operador apenas declara o limiar de tráfego, sem escrever tabela nem ação; o sistema gera o programa que descarta os fluxos que excedem esse limiar. Ajustes de parâmetro após o deploy inicial ainda podem ser feitos em tempo real via P4Runtime, sem necessidade de recompilar.

Araucaria: executar computação dentro do switch (in-network computing) melhora o desempenho, mas transforma o dispositivo em um ponto único de falha, se o switch cai, a função deixa de existir junto com ele. Dar tolerância a falhas a essa computação hoje exige reescrever manualmente, em P4, a lógica de replicação e consistência. O Araucaria propõe declarar esse requisito como uma intenção, por exemplo, uma função de sincronização com determinado número de réplicas, tolerância a falhas e nível de consistência e resolve com um compilador próprio que não reescreve a lógica original: insere um estado adicional no parser e envolve o bloco de execução original, intocado, entre uma etapa de preparação e uma de finalização. Nessa camada atuam quatro blocos padronizados: detecção de falha, replicação, coleta de estado e recuperação, sempre incluídos, com parâmetros definidos pelos requisitos da intenção. A abordagem foi validada em dois switches Tofino reais (Wedge 100BF-32X), com failover médio de 0,16 segundo (desvio padrão de 0,03s) e o compilador traduzindo 800 intenções em cerca de 0,20 segundo.

2.2 Lacunas em aberto

Foram identificadas quatro lacunas ainda sem solução consolidada, com base em [Pang et al. \(2020\)](#), [Kfoury et al. \(2021\)](#) e [Hauser et al. \(2022\)](#).

Verificação: redes baseadas em intenção prometem confirmar sozinhas que a intenção está sendo cumprida, mas a verificação atua sobre a configuração instalada e não sobre o comportamento observado. Pang aponta ainda que a verificação lógica sofre com redundância, correlação, generalização e sobreposição entre as regras, gerando erros e atrasos.

Conflito entre intenções: múltiplos operadores declaram intenções em paralelo, sem coordenação, e elas podem disputar o mesmo recurso. Soluções de detecção existem, como o Jinjing, mas restritas a recortes estreitos. Além disso, detectar o conflito não define qual intenção deve prevalecer, questão de política e não de algoritmo, e cuja resolução manual anularia o propósito do paradigma.

Geração sem consciência de recurso: um gerador automático desconhece as restrições do alvo durante a geração e produz código correto porém ineficiente. Kfoury destaca que a memória do switch é limitada e local a cada estágio do pipeline, de modo que um estágio não reaproveita memória ociosa de outro. Hauser reforça o efeito prático: a maioria dos protótipos em P4 nunca foi portada para hardware.

Reconfiguração sem interrupção: alterar entradas de tabela ocorre em tempo de execução, mas alterar o programa exige recompilar e recarregar o pipeline, causando perda de conectividade. Em uma rede baseada em intenção, cuja premissa é a readaptação frequente, esse custo compromete a autonomia pretendida.